`KamaCache` 是一个分布式缓存，但也可以直接当作本地缓存来使用

其借鉴了开源项目 `groupcache` 的实现思路，在此基础上做了拓展优化：

1. 将单独 `lru` 算法实现改成多种算法插件式选择
2. 将 `http` 通信改为 `rpc` 通信提高网络通信效率
3. 细化锁的粒度来提高并发性能
4. 实现热点互备来避免 `hot key` 频繁请求网络影响性能
5. 加入 `etcd` 进行分布式节点的监测实现节点的动态管理
6. 加入缓存过期机制，自动清理超时缓存

整体架构

代码块

```
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│  Client    │    │  Client    │    │  Client    │
└──────┬───────┘    └──────┬───────┘    └──────┬───────┘
       │                   │                   │
       │    请求分发       │                   │
       ▼                   ▼                   ▼
┌──────────────┐    ┌──────────────┐    ┌──────────────┐
│ LCache节点 │◄──►│ LCache节点 │◄──►│ LCache节点 │
└──────┬───────┘    └──────┬───────┘    └──────┬───────┘
       │                   │                   │
       └──────────┬────────┘──────────┬────────┘
                  │                   │
                  ▼                   ▼
          ┌──────────────┐    ┌──────────────┐
          │  数据源/DB   │    │   etcd     │
          └──────────────┘    └──────────────┘
```

- **Client**<font style="color:rgb(13, 13, 13);">：通过 gRPC 与缓存节点通信的客户端</font>
- **LCache节点**<font style="color:rgb(13, 13, 13);">：缓存服务节点，负责存储和管理缓存数据</font>
- **etcd**<font style="color:rgb(13, 13, 13);">：用于服务注册发现和节点协调</font>
- **数据源/DB**<font style="color:rgb(13, 13, 13);">：当缓存未命中时的数据来源</font>

<font style="color:rgb(13, 13, 13);">核心组件及其关系</font>

代码块

```
┌──────────────────────────────────────────────────────────────────────┐
│                          Client Application                          │
└──────────────────────┬──────────────────────────────────────┬────────┘
                       │                                      │
                       ▼                                      ▼
```

```
┌──────────────────────────────────────┐   ┌──────────────────────────────────────┐
│         Local Cache                  │   │     Distributed Cache                │
│                                      │   │                                      │
│  ┌────────────────────────────┐      │   │  ┌────────────────────────────┐      │
│  │                            │      │   │  │                            │      │
│  │        Group               │◀─────┼───┼──│      ClientPicker          │      │
│  │  - name                    │      │   │  │  - selfAddr                │      │
│  │  - getter                  │      │   │  │  - svcName                 │      │
│  │  - mainCache               │◀─────┼───┼──│  - consistentHash          │      │
│  │  - peers                   │      │   │  │  - clients                 │      │
│  │  - loader                  │      │   │  │                            │      │
│  │  - expiration              │      │   │  └────────────────────────────┘      │
│  │                            │      │   │                │                     │
│  └────────────────────────────┘      │   │                │                     │
│                │                     │   │                │                     │
│                ▼                     │   │                ▼                     │
│  ┌────────────────────────────┐      │   │  ┌────────────────────────────┐      │
│  │                            │      │   │  │                            │      │
│  │         Cache              │      │   │  │         Client             │      │
│  │  - store                   │      │   │  │  - addr                    │      │
│  │  - opts                    │      │   │  │  - client                  │      │
│  │  - hits/misses             │      │   │  │                            │      │
│  │                            │      │   │  └────────────────────────────┘      │
│  └────────────────────────────┘      │   │                                      │
│                │                     │   └──────────────────────────────────────┘
│                ▼                     │
│  ┌────────────────────────────┐      │                    ┌──────────────────┐
│  │                            │      │                    │                  │
│  │        ByteView            │      │                    │     etcd         │
│  │  - b []byte                │      │                    │   Registry       │
│  │                            │      │                    │                  │
│  └────────────────────────────┘      │                    └──────────────────┘
│                                      │
└──────────────────────────────────────┘
                │                                                  │
                ▼                                                  ▼
┌──────────────────────────────────────┐            ┌──────────────────────────────────────┐
│     store Package                    │            │        Server                        │
│                                      │            │  - addr                              │
│  ┌────────────────────────────┐      │            │  - svcName                           │
│  │                            │      │            │  - groups                            │
│  │      Store                 │      │            │  - grpcServer                        │
│  │     Interface              │      │            │  - etcdCli                           │
│  │                            │      │            │                                      │
│  └────────────────────────────┘      │            └──────────────────────────────────────┘
│                │                     │
│                ▼                     │            ┌──────────────────────────────────┐
│  ┌────────────────────────────┐      │            │                                  │
│  │      Store Impls           │      │            │    Protobuf API                  │
│  │  ┌──────────┐  ┌────────┐  │      │            │  - LCacheServer                  │
│  │  │  LRU2    │  │ LRU    │  │      │            │  - Request/Response              │
│  │  └──────────┘  └────────┘  │      │            │                                  │
│  └────────────────────────────┘      │            └──────────────────────────────────┘
│                                      │
└──────────────────────────────────────┘
    ┌──────────────────────────────────┐
    │     Helper Components            │
    │                                  │
    │  ┌────────────────────────────┐  │
    │  │    consistenthash          │  │
    │  │  - Map                     │  │
    │  │  - LoadBalancing           │  │
```

```
    │  └──────────────────────────┘  │
    │                                │
    │  ┌──────────────────────────┐  │
    │  │      singleflight        │  │
    │  │  - Group                 │  │
    │  │  - call                  │  │
    │  └──────────────────────────┘  │
    │                                │
    └────────────────────────────────┘
```

数据存储层

ByteView

`**<font style="color:rgb(13, 13, 13);">ByteView</font>**` <font style="color:rgb(13, 13, 13);">是缓存值的不可变视图，用于防止缓存数据被外部修改</font>

代码块

```go
type ByteView struct {
    b []byte  // 存储的实际数据
}

// 返回数据副本，而不是直接返回原始切片
func (b ByteView) ByteSLice() []byte {
    return cloneBytes(b.b)
}
```

Store 接口

<font style="color:rgb(13, 13, 13);">定义了缓存存储的抽象接口，支持多种缓存实现</font>

代码块

```go
type Store interface {
    Get(key string) (Value, bool)
    Set(key string, value Value) error
    SetWithExpiration(key string, value Value, expiration time.Duration) error
    Delete(key string) bool
    Clear()
    Len() int
    Close()
}
```

<font style="color:rgb(13, 13, 13);">LRU</font>

<font style="color:rgb(13, 13, 13);">传统的 LRU (Least Recently Used) 算法实现，基于标准库的 </font> `**<font style="color:rgb(13, 13, 13);">container/list</font>**` <font style="color:rgb(13, 13, 13);">实现双向链表</font>

代码块

```
type lruCache struct {
    mu              sync.RWMutex
    list            *list.List                // 双向链表，用于维护 LRU 顺序
    items           map[string]*list.Element // 键到链表节点的映射
    expires         map[string]time.Time      // 过期时间映射
    maxBytes        int64                     // 最大允许字节数
    usedBytes       int64                     // 当前使用的字节数
    onEvicted       func(key string, value Value)
    cleanupInterval time.Duration
    cleanupTicker   *time.Ticker
    closeCh         chan struct{}             // 用于优雅关闭清理协程
}
```

lru2

<font style="color:rgb(13, 13, 13);">两级 LRU 缓存实现，将数据分散到多个桶中，提高并发性能：</font>

代码块

```
type lru2Store struct {
    locks       []sync.Mutex  // 分段锁
    caches      [][2]*cache   // 两级缓存：[桶索引][级别]
    onEvicted   func(key string, value Value)
    cleanupTick *time.Ticker
    mask        int32         // 哈希掩码，用于快速定位桶
}
```

<font style="color:rgb(13, 13, 13);">主要特点：</font>

- <font style="color:rgb(13, 13, 13);">分段锁提高并发性能</font>
- <font style="color:rgb(13, 13, 13);">两级缓存结构，提高缓存命中率</font>
- <font style="color:rgb(13, 13, 13);">自定义双向链表实现，减少内存分配</font>
- <font style="color:rgb(13, 13, 13);">高效的哈希函数和桶映射</font>

缓存核心层

Cache

`Cache` 是底层缓存存储的封装，<font style="color:rgb(13, 13, 13);">管理底层存储实现：</font>

代码块

```
type Cache struct {
    mu          sync.RWMutex
    store       store.Store  // 底层存储实现
    opts        CacheOptions // 缓存配置选项
    hits        int64        // 缓存命中次数
    misses      int64        // 缓存未命中次数
    initialized int32        // 原子变量，标记缓存是否已初始化
    closed      int32        // 原子变量，标记缓存是否已关闭
```

```
}
```

Group

`Group` 是缓存的命名空间，提供对特定数据集合的缓存管理

代码块

```
type Group struct {
    name       string
    getter     Getter          // 数据加载接口
    mainCache  *Cache          // 本地缓存
    peers      PeerPicker      // 节点选择器
    loader     *singleflight.Group // 请求合并
    expiration time.Duration   // 缓存过期时间
    closed     int32           // 标记组是否已关闭
    stats      groupStats      // 统计信息
}
```

<font style="color:rgb(13, 13, 13);">核心功能：</font>

- <font style="color:rgb(13, 13, 13);">缓存未命中时从数据源加载</font>
- <font style="color:rgb(13, 13, 13);">防止缓存穿透和缓存击穿</font>
- <font style="color:rgb(13, 13, 13);">支持分布式节点间数据同步</font>
- <font style="color:rgb(13, 13, 13);">详细的统计信息收集</font>

<font style="color:rgb(13, 13, 13);">分布式协调层</font>

<font style="color:rgb(13, 13, 13);">一致性哈希</font>

<font style="color:rgb(13, 13, 13);">基于一致性哈希算法的节点选择实现</font>

代码块

```
type Map struct {
    mu sync.RWMutex
    config       *Config
    keys         []int        // 哈希环
    hashMap      map[int]string // 哈希环到节点的映射
    nodeReplicas map[string]int // 节点到虚拟节点数量的映射
    nodeCounts   map[string]int64 // 节点负载统计
    totalRequests int64        // 总请求数
}
```

<font style="color:rgb(13, 13, 13);">特点：</font>

- <font style="color:rgb(13, 13, 13);">支持动态添加/删除节点</font>

- <font style="color:rgb(13, 13, 13);">自动负载均衡</font>
- <font style="color:rgb(13, 13, 13);">虚拟节点机制减少数据倾斜</font>

<font style="color:rgb(13, 13, 13);">SingleFlight </font>

<font style="color:rgb(13, 13, 13);">防止缓存击穿的请求合并机制</font>

代码块

```
type Group struct {
    m sync.Map // 使用sync.Map优化并发性能
}

// 针对相同的key，保证多次调用Do()，都只会调用一次fn
func (g *Group) Do(key string, fn func() (interface{}, error)) (interface{}, error) {
    // ... 实现代码
}
```

<font style="color:rgb(13, 13, 13);">核心思想：对于同一个key的并发请求，只执行一次实际的加载操作，其他请求共享结果。</font>

<font style="color:rgb(13, 13, 13);">服务注册与发现</font>

<font style="color:rgb(13, 13, 13);">基于 etcd 的服务注册与发现</font>

网络通信层

grpc 客户端

gprc 服务端

<font style="color:rgb(13, 13, 13);">设计模式与最佳实践</font>

<font style="color:rgb(13, 13, 13);">使用的设计模式</font>

- **单例模式**<font style="color:rgb(13, 13, 13);">：全局缓存组注册表</font>
- **工厂方法**<font style="color:rgb(13, 13, 13);">：创建不同类型的缓存存储</font>
- **策略模式**<font style="color:rgb(13, 13, 13);">：不同的缓存淘汰算法</font>
- **代理模式**<font style="color:rgb(13, 13, 13);">：客户端代理远程缓存操作</font>
- **组合模式**<font style="color:rgb(13, 13, 13);">：缓存组织结构</font>

<font style="color:rgb(13, 13, 13);">性能优化技巧</font>

- **预分配内存**<font style="color:rgb(13, 13, 13);">：减少动态内存分配</font>
- **批量操作**<font style="color:rgb(13, 13, 13);">：减少系统调用</font>
- **分段锁**<font style="color:rgb(13, 13, 13);">：减少锁竞争</font>
- **异步处理**<font style="color:rgb(13, 13, 13);">：非关键路径使用异步操作</font>
- **惰性删除**<font style="color:rgb(13, 13, 13);">：标记删除而非立即物理删除</font>

<font style="color:rgb(13, 13, 13);">扩展性设计</font>

- **接口抽象**<font style="color:rgb(13, 13, 13);">：Store、Getter、PeerPicker等接口</font>
- **选项模式**<font style="color:rgb(13, 13, 13);">：使用函数选项模式配置组件</font>
- **中间件思想**<font style="color:rgb(13, 13, 13);">：可插拔的组件设计</font>